arXiv:2603.10726v2 [cs.CR] 20 May 2026

PrefixWall: Mitigating Prefix Caching Side Channels in Shared LLM Systems

Panagiotis Georgios Pennas
IMDEA Software Institute
Universidad Politécnica de Madrid
panagiotis.pennas@imdea.org

Konstantinos Papaioannou
IMDEA Software Institute
Universidad Politécnica de Madrid
konstantinos.papaioannou@imdea.org

Marco Guarnieri
IMDEA Software Institute
marco.guarnieri@imdea.org

Thaleia Dimitra Doudali
IMDEA Software Institute
thaleia.doudali@imdea.org

ABSTRACT

Large Language Models (LLMs) rely on optimizations like Automatic Prefix Caching (APC) to accelerate inference. APC works by reusing previously computed states for the beginning part of a request (prefix), when another request starts with the same text. While APC improves throughput, it introduces timing side channels: cache hits are faster than misses, creating observable latency differences. In multi-tenant systems, attackers can exploit these differences to infer sensitive information, e.g., by incrementally reconstructing another user's request by observing hit/miss patterns. Current defenses take a sledgehammer approach: they disable APC and cache sharing, isolating users, and sacrificing efficiency for regular users. This paper presents PrefixWall, a system that secures multi-tenant LLM serving systems against APC side channels without sacrificing performance and efficiency. PrefixWall monitors cache reuse across users, flags suspicious sharing, and selectively isolates prefixes, restricting their reuse, only when necessary. Evaluation shows that PrefixWall enables up to 70% higher cache reuse and 30% lower inference latency compared to existing defenses that isolate users. PrefixWall's lightweight design demonstrates how security in LLM serving does not have to come at the cost of unnecessarily reduced performance or unbearable overheads.

1 INTRODUCTION

Large Language Models (LLMs) now power applications such as conversational assistants, code generation, and enterprise analytics [13]. These services operate at massive scale under strict latency and throughput requirements, making inference serving a critical systems challenge [76]. Modern serving systems employ system-level optimizations such as cache management [39, 40, 59] and scheduling [2, 3, 33] to accelerate LLM inference. Among these optimizations, prefix sharing [10, 37, 71], or otherwise known as *Automatic Prefix Caching (APC)*, is widely used by state-of-the-art frameworks and commercial APIs, such as OpenAI [49], DeepSeek [19], Google Gemini [18], MoonShot Kimi [55], vLLM [39], and SGLang [74]. APC accelerates inference by caching and reusing previously computed model states for the beginning part of a request (the request's *prefix*) whenever another request starts with the same text. The use of a *prefix cache* avoids redundant computation and significantly reduces latency for long prompts and multi-turn conversations [37, 71].

While APC delivers substantial performance benefits, it also introduces security risks similar to other caching optimizations [12, 17, 30, 70]. APC creates observable variations in inference latency depending on whether a request reuses cached prefixes or not. These latency differences create a *timing side channel*, that is, an unintended information leak where execution time reveals properties of secret data without direct access. In multi-tenant deployments where the prefix cache is shared across different security domains (e.g., different users), attackers can exploit this side channel to infer private information. For example, by sending crafted requests and measuring time-to-first-token (TTFT), an attacker can determine whether parts of their input match cached prefixes from another user's request. Repeating this process enables prompt-stealing attacks that reconstruct sensitive content from other users [31, 58].

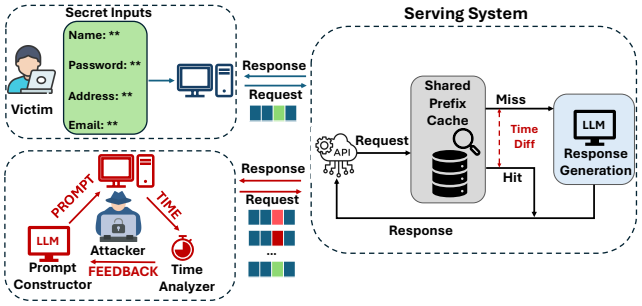


Figure 1: Timing side-channel leakage in prefix-sharing LLM inference. The attacker sends crafted prompts and measures time-to-first-token (TTFT) to detect cache hits or misses caused by Automatic Prefix Caching (APC) and steal the sensitive information in the victim's prompt.

To prevent these attacks, prior work has proposed several defenses against APC timing leaks [14, 16, 31, 51, 75]. These approaches span three main strategies: (i) full user-level isolation, which disables prefix sharing entirely [31, 51, 75]; (ii) timing obfuscation, which injects noise to mask latency differences [14]; and (iii) selective isolation of secret-dependent prompts based on LLM-aided semantic analysis [16]. While effective in theory, all these defenses suffer from a critical drawback: they introduce significant overhead, *often unnecessarily*. For example, for prompts that are not involved in attacks or for situations where the timing side channel is not exploitable in practice. As a consequence, these solutions erase APC's performance benefits and penalize regular, benign users;

thereby limiting their applicability in realistic, high-performance multi-tenant environments.

To address the shortcomings of current defenses, we introduce **PrefixWall**, a system-level solution that secures multi-tenant LLM serving against timing side channels caused by APC, while preserving its performance benefits. PrefixWall is based on the **key insight** that isolating entire users (as done in [31, 51, 75]) is not necessary to prevent attacks; rather *it is sufficient to protect the shared prefixes that might lead to attacks*. Building on this insight, PrefixWall (1) continuously monitors prefix reuse across users, (2) flags prefixes that are suspiciously reused by multiple users, and (3) prevents further reuse of flagged prefixes in follow-up requests. This approach enables efficient sharing for benign users, while selectively isolating only those prefixes associated with suspicious behavior, avoiding unnecessary penalties for all users.

The system design of PrefixWall consists of lightweight components that work together to secure prefix caching without sacrificing efficiency. First, PrefixWall extends the prefix cache with minimal metadata for each entry to track ownership and flag suspicious reuse. Second, the *Detector* monitors cache hits and isolates flagged prefixes when multiple users attempt to share them. Finally, PrefixWall introduces a system-level optimization that activates prefix isolation only when a timing side channel is actually exploitable. This is handled by the *Activator*, which continuously evaluates whether latency differences between cache hits and misses are distinguishable, based on factors such as the hardware platform, LLM model size, system load, and request length. These parameters directly influence the strength and exploitability of the timing side channel, as demonstrated in our motivational analysis. Overall PrefixWall's system components interact seamlessly: the cache extension provides metadata, the Detector enforces selective isolation, and the Activator optimizes the security-performance trade-off.

PrefixWall is implemented on top of the open-source, state-of-the-art LLM serving system vLLM [64]. We conduct an extensive evaluation across multi-tenant workloads with varying levels of intra- and inter-user prefix sharing and nine LLM models spanning different families and sizes (from 0.5B to 13B parameters). Our experimental results show that PrefixWall achieves up to 70% higher cache reuse and 30% lower inference latency compared to defenses that enforce user-level isolation. In addition, PrefixWall introduces negligible time and memory overheads during runtime, demonstrating that security can be achieved without sacrificing performance when implemented in a lightweight and principled way. Finally, to validate the security of PrefixWall, we performed a detailed security analysis that precisely characterizes PrefixWall's security guarantees, which we complement with an empirical validation showing that PrefixWall indeed closes the timing side-channel introduced by APC across requests from different users.

The specific **paper contributions** are:

- A detailed analysis of timing side channels introduced by Automatic Prefix Caching (APC) in multi-tenant LLM serving systems, including observations on their exploitability under different workload and system conditions (Section 2).
- The design of **PrefixWall**, a lightweight and practical system that secures prefix reuse against timing attacks by selectively isolating suspicious prefixes rather than entire users, while preserving performance benefits (Section 3). PrefixWall will be **open-sourced** to allow community adoption.
- A comprehensive security analysis of PrefixWall 's guarantees and limitations, complemented by empirical validation against prompt-stealing attacks (Section 4).
- An extensive evaluation of PrefixWall across diverse workloads and state-of-the-art LLM models, demonstrating up to 70% higher cache reuse and 30% lower latency compared to user-level isolation defenses, with negligible overhead (Section 5).

2 BACKGROUND AND MOTIVATION

In this section, we provide background and motivation for PrefixWall. We start by introducing background on LLM serving systems and, in particular, on the APC optimization (2.1). We then characterize the parameters that influence the exploitability of timing side channels due to APC (Section 2.2). Then, we illustrate how attackers can exploit APC through timing leaks to learn sensitive prompts (2.3). Next, we overview current defenses for APC timing leaks and discuss their limitations (2.4). We conclude by summarizing our motivational observations in (2.5).

2.1 Caching in LLM Serving Systems

Large Language Models (LLMs), such as GPT [13] and LLaMA [62], are the backbone of modern AI services. Integrating them in modern IT systems requires serving systems that deliver inference with low latency and high throughput under multi-user workloads. Inference happens in two stages: *prefill* and *decode*. During prefill, the entire input prompt (also called a *request*) is processed in a single forward pass, generating key-value (KV) tensors for each token. In the decode phase, tokens are produced autoregressively; for each new token, the model computes a new key-value tensor using all previously generated tensors. Without caching, every decode step would recompute all KV tensors for both prefill and prior decode tokens, incurring quadratic cost.

To avoid this, current LLM serving systems employ a KV cache that stores tensors generated during the prefill and previous decoding steps for reuse. To further accelerate inference, several commercial frameworks such as OpenAI [49], DeepSeek [19], Google Gemini [18], MoonShot Kimi [55], vLLM [39], and SGLang [74] implement *Automatic Prefix Caching (APC)*. APC reduces redundant computation by reusing cached KV tensors whenever a new request shares a prefix with an earlier one. This optimization is particularly effective for scenarios such as long-document queries or multi-round conversations [65], where repeated processing of the same prefix would otherwise incur significant overhead.

When serving systems apply APC, a prompt can be viewed as *a sequence of prefixes*, each mapped to a cache entry in the KV/prefix cache. Requests may experience partial cache hits, starting from the beginning of the prompt and reusing some cached prefixes while recomputing others. The leftmost example in Figure 5 illustrates this process, where each node corresponds to a cache entry for that particular prefix/part of the prompt sentence.

2.2 Timing Differences due to APC

Although the use of the prefix cache accelerates inference, it introduces observable latency differences between cache hits and misses, particularly in the time-to-first-token (TTFT). These variations create a *timing side channel*, an unintended information leak where execution time reveals whether a request shares prefixes with cached prompts. In multi-tenant environments, such leaks can be exploited by attackers to infer sensitive information. In the following examples, we illustrate how APC-induced timing differences manifest and we identify the *key parameters* that influence the strength and exploitability of the side channel.

Example 2.1. To illustrate how APC impacts latency, recall from Section 2.1 that a prompt is processed as a sequence of prefixes, each mapped to a cache entry. When APC is enabled, these prefixes may either hit in the cache or require recomputation. We consider two cases: one where all prefixes of the prompt hit the cache, and another where a miss occurs early, forcing all subsequent prefixes to be recomputed. This difference in reuse leads to observable variations in time-to-first-token (TTFT) between requests that fully reuse cached prefixes and those that recompute them. 2 reports TTFT comparisons across four LLMs of increasing size and varying prompt lengths, with requests sent at a constant rate (RPS = 1). TTFT for cache hits is shown in red, while misses are shown in blue. For all models except the smallest one (i.e., LLava-0.5B), we highlight the following aspects:

- TTFT differences between cache hits and misses become noticeable after a certain prefix length for each model.
- The latency gap grows as the shared prefix length increases.
- For the same prefix length, larger LLMs exhibit greater TTFT differences.
- Larger LLMs show distinguishable timing differences even at shorter prefixes.

For LLava-0.5B, TTFT differences between hits and misses are negligible because the model is small and the recomputation latency for the misses is minimal.

Observation 1: TTFT reveals whether a request reuses cached prefixes (hits) or recomputes them (misses), and the difference becomes more pronounced with larger models and longer prefixes that are reused across users.

Example 2.2. Next, we examine how system load influences these timing differences using the same setup as before with the Llama-13B model. 2 shows TTFT measurements under increasing request-per-second (RPS) rates (10, 20, 30, 40) for prompts with prefixes of 100–500 tokens. We observe that as the request-per-second (RPS) rate increases, the latency gap between cache hits and misses progressively collapses, making them hard to distinguish. This happens because higher RPS introduces resource contention and head-of-line blocking in the LLM serving pipeline [2, 3, 52, 76]. Under these conditions, batching and queuing delays dominate latency, overshadowing the computational savings of prefix reuse. Consequently, cache hits appear as slow as misses, effectively eliminating any timing differences.

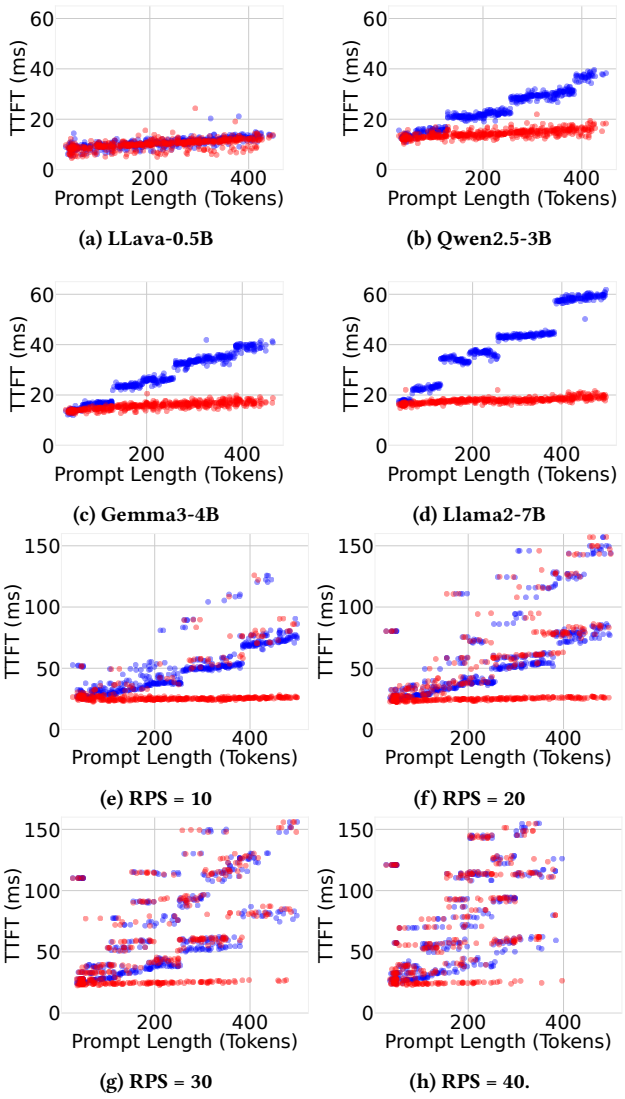


Figure 2: TTFT difference between cache hits (red) and misses (blue) for increasing length of prefixes/prompts reused across users. Examples for different LLM models and system load (requests per second RPS).

Observation 2: As system load increases, TTFT differences between prefix cache hits and misses disappear because batching and queuing delays dominate latency, masking cache-related timing variations.

To further validate observations 1 and 2 across a wide range of conditions, Figure 3 shows the KDE overlap between the TTFT distributions of cache hits and misses across 5 different LLMs, increasing prefix/prompt length and system load (requests per second). KDE overlap is a statistical measure of similarity between two probability distributions, computed as the integral of the minimum of their density functions [34]. In our context, it quantifies

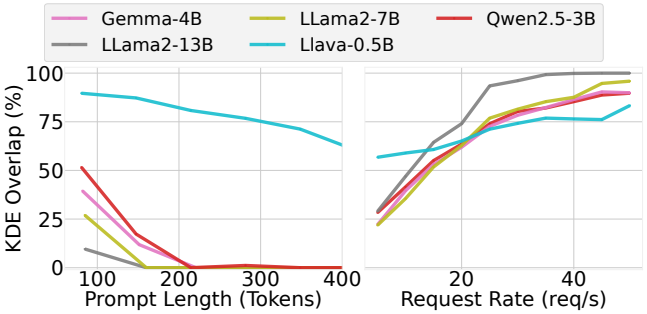


Figure 3: Effect of the LLM model, prefix/prompt length and system load (requests per second) on the distinguishability of the APC timing differences, which is captured with the KDE overlap between cache hits and misses.

how much the TTFT distributions of cache hits and misses intersect: higher overlap indicates that hits and misses are harder to distinguish, while lower overlap means the opposite. The left plot shows that the overlap decreases as the prefix length grows, making timing differences more pronounced for longer prefixes, especially in larger models such as LLaMA-13B and LLaMA-7B. In contrast, smaller models like Llava-0.5B maintain high overlap even for long prefixes, meaning that it's hard to distinguish any TTFT differences, as shown previously in Figure 2. The right plot shows that overlap increases with request rate (RPS), as batching and queuing delays dominate latency and mask cache-dependent variations. At high RPS, hits and misses become nearly indistinguishable across all models, effectively eliminating the timing leak.

In conclusion, while prior work acknowledges APC-induced timing side channels [16, 31, 51, 75], our motivational analysis shows that the strength and exploitability of the channel depend on the following critical parameters:

- **the length of the shared prefix:** longer prefixes are more vulnerable.
- **the size of the LLM model:** larger models are more vulnerable.
- **the system load**: lower load (requests received per second) is more vulnerable.
- **the underlying hardware platform:** since it directly influences timing measurements.

These parameters jointly determine whether timing side channels are distinguishable and, should be taken into consideration into building robust and practical mitigation mechanisms.

Key insight: While APC introduces timing side channels, their exploitability depends on various parameters: the prefix length, LLM size, system load, and hardware platform. Prior work treated APC leaks as uniformly exploitable, whereas our analysis reveals that these parameters jointly determine side-channel strength and must guide efficient and practical mitigation.

2.3 Prompt Stealing Attacks via APC leaks

In multi-tenant systems the prefix cache is shared between requests from different users and security domains. Hence, these visible timing differences can be exploited by attackers to incrementally reconstruct sensitive prompts or steal private information (secrets) of other users by observing the TTFT of attacker requests. That is, APC leaks can be exploited to perform *prompt stealing attacks* [58]. Prior studies [16] show that datasets capturing real-world LLM workloads, such as ShareGPT [9], Multiturn Chat [11], and Prompt Multitasks [24], reveal frequent prefix reuse within and across users. Specifically, requests from the same user share prefixes in 9–60% of cases, while up to 30% reuse occurs across different users. Thus, APC leaks are a real threat for the security of multi-tenant LLM serving systems.

Figure 1 illustrates the workflow of a timing-based prompt stealing attack that exploits APC leaks, inspired by [58]. First, the victim user issues a request that includes a prompt containing sensitive information. For simplicity, here we assume that the prompt follows a fixed template (known by the attacker) and it includes user-specific secrets. For example, a prompt template can be *Compose a meeting agenda for an interdisciplinary team discussing the treatment plan for [Name] with [medical condition]*, where *[Name]* and *[medical condition]* are secret. The attacker uses an LLM-based *Prompt Constructor* to create candidate prompts by varying words within the template. Next, each candidate is submitted to the LLM serving system, and the time-to-first-token (TTFT) is measured. By observing the TTFT, the attacker can determine (e.g., using a dedicated pre-trained classifier) whether the observed latency corresponds to hits or misses in the prefix cache. In particular, a cache hit indicates that the candidate shares a prefix with the victim's prompt. By repeating this process iteratively, the attacker can reconstruct the original prompt word-by-word or retrieve missing/private data fields (e.g., names, dates, identifiers) without direct access to the cache [31, 58].

2.4 Defenses Against APC leaks

Recent works have proposed several defenses against timing leaks induced by APC. These defenses fall into three main categories, each with distinct trade-offs between security and performance, as summarized in Table 1.

Cache Isolation and Partitioning. On the one extreme, several works propose mitigating APC leaks by eliminating cache sharing entirely across different users and security domains. InputSnatch [75] and Auditing Prompt Caching [31] advocate per-user cache isolation or disabling prefix reuse across requests. Cache Partitioning [51] similarly enforces user-boundaries within the KV-cache to prevent cross-user hits. These approaches ensure strong security guarantees and effectively block timing leaks due to shared prefixes by addressing the leak at its source. However, they also sacrifice the performance benefits of prefix caching, which can result in significant overhead given that "real-world LLM workloads frequently exhibit significant cross-query reuse" [16].

Shortcoming 1: User-based cache isolation prevents timing leaks but introduces overhead due to reduced prefix sharing.

Timing Obfuscation. On the other extreme, Carlini and Nasr [14] propose to secure systems by making the execution time of inference "constant" across hits and misses. For this, they propose to inject noise in the system to mask timing signals and, ultimately,

Related Work	Mitigation	Prefix Reuse	Low TTFT
Partitioning [51]	Per-user Isolation	✗	✗
InputSnatch [75]	Per-user Isolation	✗	✗
Audit [31]	Per-user Isolation	✗	✗
Remote [14]	Timing Obfuscation	✓	✗
SafeKV [16]	Selective Sharing	✓	✗
PrefixWall	Selective Sharing	✓	✓

Table 1: Comparison of current mitigation strategies.

close the side channel by making hits and misses in the prefix cache indistinguishable to attackers. That is, these defenses would ensure that the distributions of hits and misses in 2 overlap. Although obfuscation defenses can prevent APC leaks, they have two limitations. First, they introduce uniform delays across all requests, penalizing benign users and reducing overall performance. Second, they do not address the source of the timing leak (i.e., the prefix cache being shared between security domains) and historically obfuscation defenses for timing leaks are less robust since attackers can often find ways of amplifying even small differences in execution time [29, 38, 63].

Shortcoming 2: Obfuscation-based techniques do not address the source of leaks and introduce overheads for benign requests.

Selective Cache Sharing. To balance security and efficiency, some defenses restrict cache sharing rather than disabling it completely. SafeKV [16] introduces a multi-tier detection pipeline that semantically classifies requests using rule-based checks and semantic validation via LLMs either as "sensitive" (i.e., that may contain secret information) or non-sensitive. In SafeKV, only requests classified as sensitive are isolated at cache-level, thereby preventing other users (attackers included) from hitting on them. Although this approach prevents leakage without fully disabling APC, its reliance on heavyweight semantic LLM-based analysis limits scalability for high-performance deployments. Moreover, some requests might be misclassified and, therefore, left unprotected.

Shortcoming 3: Current selective sharing defenses may fail in preventing leaks for "misclassified" requests and still introduce unnecessary overhead.

2.5 Summary

Our motivational analysis reveals that Automatic Prefix Caching (APC), while critical for high-throughput LLM inference, introduces timing side channels whose exploitability depends on concrete parameters such as the prefix length, LLM size, system load and, effectively, the underlying hardware platform. Longer prefixes that are reused across users and larger models amplify latency gaps between cache hits and misses, enabling adversaries to infer sensitive prompts under low-load conditions. Existing defenses fail to reconcile this tension between security and efficiency: isolation-based approaches eliminate cache sharing entirely, sacrificing the performance benefits that make APC essential; obfuscation techniques inject noise without addressing the root cause, imposing uniform delays and remaining vulnerable to amplification attacks; and selective sharing mechanisms rely on heavyweight LLM-based semantic analysis, limiting scalability and leaving misclassified requests exposed. These shortcomings highlight **a fundamental gap:** we need a defense that *adapts to the conditions under which timing leaks are exploitable*, secures prefix reuse without blind per-user isolation, and preserves the efficiency of APC across diverse workloads, without introducing unbearable overheads in return for security. This motivates the design of a new system that achieves robust security with lightweight, practical mechanisms, enabling multi-tenant LLM serving to remain both performant and secure.

3 SYSTEM DESIGN

In this section, we introduce PrefixWall, that provides system-level security against timing leaks induced by APC. We start by providing a high-level overview of PrefixWall and its objectives (3.1). Next, we describe PrefixWall's core components (Sections 3.2 and 3.3).

3.1 Overview and Objectives

PrefixWall deploys a lightweight defense mechanism that secures multi-tenant LLM serving systems against timing side-channel attacks caused by Automatic Prefix Caching (APC). PrefixWall builds on the **key insight** that preventing timing side-channel attacks does not require isolating entire users (like it is done in [31, 51, 75]); it only requires isolating *prefixes* that might lead to attacks. Building on this insight, PrefixWall allows cached prefixes to be reused across users but stops reuse beyond prefixes that could reveal sensitive information. This selective prefix isolation works on a simple principle: attacks require different users to share and reuse a prefix. PrefixWall enforces this by tracking how users interact with shared cache entries without requiring any knowledge of prompt semantics [16]. Benign users benefit from shared caching, where common prefixes remain widely reusable. When suspicious behavior is detected, PrefixWall intervenes by isolating the corresponding prefixes rather than penalizing all users. This design ensures that cache reuse within and across users is maximized to the greatest extent possible without compromising security.

Objectives. The design of PrefixWall is guided by the following three objectives:

- **[O1] Secure Prefix Reuse:** Prevent timing attacks introduced by Automatic Prefix Caching (APC) by isolating suspicious prefixes, avoiding heavy-handed approaches such as per-user isolation or disabling cache sharing.
- **[O2] Performance Preservation:** Maximize cache reuse and accelerate inference speed, across diverse workloads and LLM models, thereby ensuring that security does not come at the cost of performance.
- **[O3] Lightweight and Practical Design:** Provide real-time protection with minimal overhead, without requiring semantic analysis or prior knowledge of sensitive content, and integrate seamlessly with state-of-the-art LLM serving systems.

System overview. To achieve these objectives, PrefixWall introduces a lightweight pipeline that operates on the request-serving path, as illustrated in Figure 4. PrefixWall extends the KV cache with minimal metadata per entry to enable fine-grained tracking of user-prompt interactions and flag suspicious prefixes that are

shared and reused across users. When a request arrives, the *Activator* first decides whether selective isolation should be enabled based on the distinguishability of the timing side channel, under the current LLM model, prefix length, system load and underlying hardware platform. At the same time, the LLM serving system checks whether the prefixes that are part of the request hit or miss the cache. On a cache miss, a new cache entry is created and tagged with the user's ID; no further action is needed because misses do not leak timing information.

On a cache hit, the *Detector* determines whether the prefix should be isolated to prevent attacks or shared as usual, following the procedure described in Section 3.2. If the prefix is shared (marked as 3 in Figure 4), the request proceeds with full reuse and the response is returned directly to the user. If the prefix is isolated, PrefixWall stops reuse at that point and recomputes the remaining tokens through the LLM, creating a new cache entry for the isolated prefix. This results in a *partial hit*: the request benefits from reuse up to the shared prefix, while the remaining portion is isolated and recomputed to prevent leakage.

PrefixWall consists of the following system components:

- **KV Cache Extension:** Augments each cache entry with metadata to track user-prompt interactions and flag suspicious prefixes (Section 3.2).
- **Detector:** Examines prefix reuse to identify cross-user access and enforce selective isolation when suspicious reuse occurs (Section 3.2).
- **Activator:** Dynamically activates selective isolation only under conditions that create exploitable timing side-channels, balancing security with performance (Section 3.3).

Together, these components enable PrefixWall to maximize secure cache reuse while mitigating timing attacks without sacrificing efficiency.

3.2 KV Cache Extension and Detector

3.2.1 KV Cache Extension. PrefixWall extends each cache entry with the following metadata fields:

- **OwnerID**: the identifier of the user who first created and populated this cache entry. It is set exactly once at allocation time and remains immutable.
- **AttackFlag**: a marker indicating that this prefix has been flagged for isolation due to being reused by multiple users. When set, prefix reuse beyond this point must be disabled for non-owners.

These fields add only a few bytes per entry and do not affect tensor layout or memory mapping. Next, we explain why we track these metadata.

Why Track the Owner? Tracking `OwnerID` is essential because any user other than the original creator is a potential attacker and may attempt to infer information about private fields of a prefix by exploiting timing differences. By recording ownership, PrefixWall can distinguish benign reuse from suspicious one and enforce selective isolation only when necessary.

Why Flag a Cache Entry? Flagging an entry is essential because cross-user reuse is the point where timing side channels become exploitable. When a request reuses cache entries created by another user, the risk of prompt stealing begins. By flagging a suspicious cache entry, PrefixWall sets a boundary: reuse up to this point is allowed, but going further is restricted for non-owners (potential attackers).

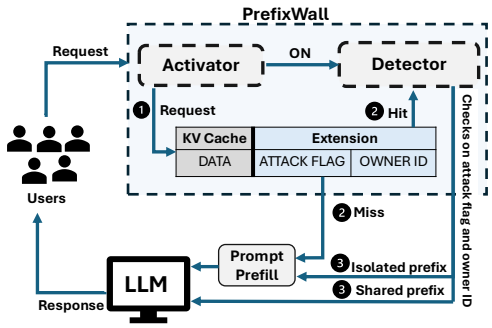


Figure 4: System Design of PrefixWall.

3.2.2 Detection Pipeline. When a cache entry is created on a cache miss, its metadata is initialized: `OwnerID` is set to the current user ID and `AttackFlag` is cleared. When there is a cache hit, the request is sent to the *Detector* for validation. At this stage, the Detector examines whether the prefix should remain shared or be isolated to prevent leakage. Two scenarios can occur during validation:

- **Hit on an unflagged prefix:** If the prefix is unflagged and owned by the requesting user, reuse proceeds without changes, the cached entry is returned back to the user. If the prefix is owned by a different user, the Detector flags this prefix as isolated for future requests, ensuring that reuse beyond this point is blocked for non-owners.
- **Hit on a flagged prefix:** If the prefix is previously flagged, the Detector checks whether the next prefix in the prompt belongs to the requesting user. If yes, reuse continues normally. If not, reuse stops at the flagged prefix, and the remaining ones are recomputed through the LLM.

3.2.3 Example Workflow. Figure 5 illustrates how PrefixWall handles reuse across three users that issue requests at consecutive timestamps t_1–t_4. Each request contains similar prompts that differ only on private data, such as names. In the figure, circles represents cache entries, and arrows show the sequence of prefixes in a prompt. Red circles indicate cache misses, green circles indicate cache hits, and orange circles mark flagged entries where selective isolation applies. Next, we describe what happens at each timestamp:

t_1: User 1 submits a prompt and populates the cache with a new cache entries tagged with `OwnerID = 1`. All entries are unflagged.

t_2: User 2 issues a prompt that reuses the first two entries from User 1. During detection, PrefixWall checks the `OwnerID` of every entry and identifies that the second entry belongs to a different user. This entry is then flagged by setting its `AttackFlag` (orange in the figure). New and separate cache entries are created beyond this flagged entry for User 2.

t_3: User 1 sends a follow-up prompt. Although the flagged entry is reused, this is acceptable because User 1 is the original owner. Thus, PrefixWall allows User 1 to reuse all prefixes beyond the flagged entry without restriction (owner-aware continuation).

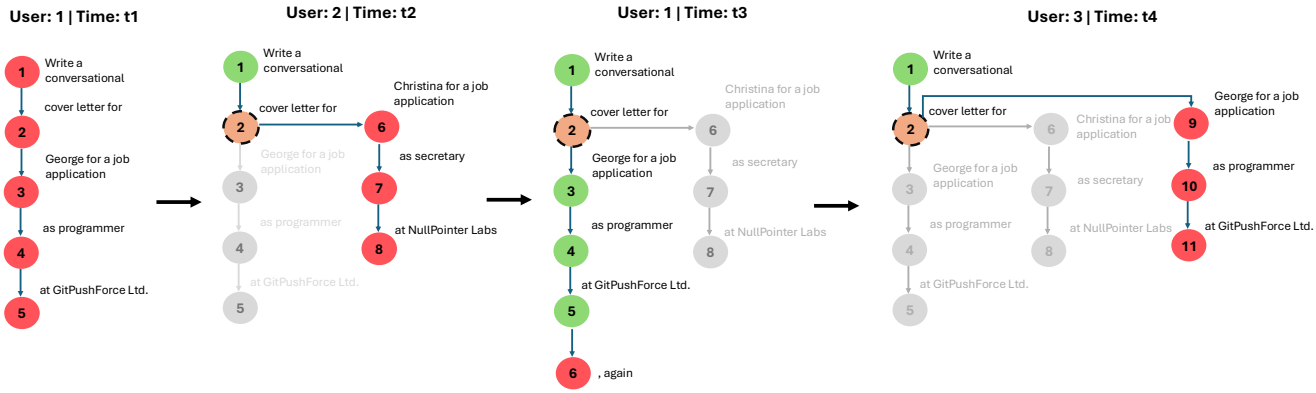


Figure 5: Example workflow of PrefixWall.

t_4: User 3 attempts to reuse User 1's prompt but with a different name in the private information. PrefixWall detects that the `AttackFlag` is set and that the `OwnerID` differs, so continuation is not allowed. The common prefix is isolated at the flagged entry to prevent leaks, and new cache entries are created for User 3.

In summary, this example illustrates PrefixWall's selective isolation: flagged cache entries restrict cross-user continuation, diverging into new secure paths for non-owners, while allowing full reuse along the original path for the owner. We stress that PrefixWall always allows owners to extend their prefixes even though some nodes might be flagged to ensure that benign requests are not delayed. For instance, User 2 would be allowed to further extend their own path 1-2-6-7-8 although node 2 is flagged.

3.3 Activator Optimization

As shown in Section 2.2, timing side channels due to APC are *conditionally* distinguishable and exploitable: latency gaps between cache hits and misses grow with (i) longer shared prefixes and (ii) larger model sizes, but are (iii) progressively masked as request-per-second (RPS) increases due to batching, queuing, and GPU saturation effects, (iv) while the hardware platform directly influences timing differences. Therefore, a defense that is always-on would over-protect under high load and impose unnecessary overhead on benign users. PrefixWall addresses this by allowing system administrator to conditionally *activate* selective isolation only when timing signals are distinguishable for the current operating point (model, hardware, workload).

To achieve this, PrefixWall continuously monitors the latency gap between hits and misses over a sliding time window and computes the *Kernel Density Estimation (KDE) overlap* between their distributions. The KDE overlap [34] is a statistical measure of similarity that indicates how distinguishable two distributions are. Values close to 100% mean the two distributions are nearly identical (i.e., hits and misses are statistically indistinguishable), while values near 0% mean they are well separated (i.e., an attacker can easily tell them apart).

When the overlap exceeds a threshold θ, PrefixWall deactivates selective isolation and allows full prefix reuse. However, the cache is still updated with the extra metadata. Conversely, when the overlap falls below θ, timing differences are exploitable, and PrefixWall activates its detection and prefix isolation mechnism. This decision is evaluated at every request using the most recent sliding window of TTFT samples, ensuring that activation adapts dynamically to workload conditions. The selection of the threshold θ is left to the system administrator and it should reflect (1) the point where hits and misses are statistically indistinguishable, which would make timing attacks impractical, and (2) the trade-off between stronger security and performance overhead.

4 SECURITY ANALYSIS

In this section, we analyze the security guarantees provided by PrefixWall. We start our analysis by focusing on the case where PrefixWall is activated (3.2), and later on we analyze the impact of the activator (3.3) on the security of the system.

Security guarantees when PrefixWall is activated. PrefixWall's selective isolation scheme has been designed to provide security against prompt-stealing attacks performed through APC timing side channels. In this setting, an attacker attempts to reconstruct a prompt that contains some secrets information. The prompt reconstruction works by discovering one entry at-a-time and can be split into multiple *entry-reconstruction problems*, each one focusing on discovering the next secret entry *secret* given a known prefix *pre*. We analyze PrefixWall's security guarantees in terms of one of these sub-problems.

Let $pre \cdot secret$ be a prefix where *pre* is a known part extended with a secret entry *secret*. To reconstruct the secret entry, an attacker (1) issues a *probing sequence* $pre \cdot v_1, \ldots, pre \cdot v_n$ consisting of many different requests $pre \cdot v_i$ sharing a common prefix *pre* and differing in the last entry, while (2) measuring the TTFT of the probes to determine whether they results in hits or misses in the prefix cache.

To prevent these timing attacks, PrefixWall identifies prefixes that are shared between different users and selectively isolates them, which stops an attacker's probing sequences from leaking

information about a victim's prefix. Below, we precisely characterize the guarantees provided by PrefixWall's isolation scheme.

Security guarantees: Consider an arbitrary initial state of the KV-cache c containing a secret prefix $pre \cdot secret$ owned by user u and a probing sequence $\overline{s} := pre \cdot v_1, \ldots, pre \cdot v_n$ (for different values of v_i). If (1) $|pre| \geq 1$, and (2) either pre's final entry in c is already flagged or $v_1 \neq secret$, then PrefixWall ensures that executing all probes in the sequence $\overline{s}$ from user identifiers different from u (even interleaved with additional disjoint requests) will always result in misses for entries $v_1, \ldots, v_n$.

We note that PrefixWall's security guarantees are not unconditional, that is, PrefixWall protects against probing sequences only when conditions (1) and (2) above are met. We remark that this is a conscious choice made as part of PrefixWall's trade-off between security and performance. Next, we describe the two cases not covered by PrefixWall:

- **Attacks targeting the first entry:** PrefixWall does not protect against prompt-stealing attacks targeting the first entry of a prompt, i.e., against cases where there is no (non-empty) prefix (that is, condition (1) above). This follows from the fact that in this case there is no parent node that PrefixWall can flag as potentially involved in an attack. However, as we show in 2.2, the security risks associated with attacks targeting prefixes consisting of a single entry is limited, since even for the largest model we consider the distributions of hits and misses overlap (2d), which makes the timing leak more difficult to exploit. Furthermore, in real world deployments prefix caching is often only enabled for longer prefixes, e.g., OpenAI enables prefix reuse only for prompts longer than 1024 tokens [49].
- **Attacks correct at the first attempt:** PrefixWall does not protect an *unflagged* prefix $pre \cdot secret$ against an attacker that guesses correctly the secret value on the first attempt (i.e., condition (2) above). This is due, again, to the lack of a flagged prefix pre that is needed for PrefixWall to start isolating a request. From a security perspective, this can be problematic if the space of secrets is very small since an attacker might easily guess correctly on the first attempt, and the risk decreases for larger secret spaces. Note that preventing these attacks by always treating a prefix as flagged would make PrefixWall fall back to naive user-level isolation, which introduces too high overhead.

Beyond the two cases mentioned above, PrefixWall successfully secures the system against all remaining probing sequences. We remark that PrefixWall's guarantees do not depend on identifying secret information at a semantic level (like in [16]), but rather on the overlap between prefixes from different users and security domains. Furthermore, PrefixWall's guarantees hold regardless of (1) whether a probing sequence is executed by a single attacker or multiple colluding ones, (2) whether benign requests from other users are interleaved in the probing sequence (since these requests at most would result in evicting the victim prefix from the prefix cache), and (3) the initial state of the prefix cache.

Security impact of threshold-based activation. The activator (3.3) turns off PrefixWall's selective-isolation scheme whenever exploiting the timing side-channel is impractical (captured by the KDE threshold specified by the administrator). This can impact the security of the system in two ways. If the system administrator chooses a too-low overlap threshold, this may results in the system being unprotected even when the attacker can effectively exploit the timing side-channel induced by APC. Even if the overlap threshold is properly chosen (i.e., the system is disabled only when the signal-to-noise ratio in the side-channel is very low), an attacker might still attempt to bias the activation logic by shaping traffic patterns. For example, issuing bursts of requests to inflate the KDE overlap and force deactivation, or throttling requests to reduce overlap and trigger isolation for probing. PrefixWall mitigates this risk through two design choices. First, the Activator computes the KDE overlap over a sliding window of recent requests, requiring sustained conditions rather than reacting to instantaneous fluctuations. Second, activation decisions rely on aggregate system-level measurements rather than per-user metrics, making it difficult for a single attacker to dominate the observed distributions. Together, these mechanisms ensure that activation cannot be easily manipulated by short-term traffic manipulation. Furthermore, we remark that even when selective isolation is disabled, PrefixWall still tracks and updates the metadata in the cache. That is, prefixes that are shared between different users are still flagged in the prefix cache even when the isolation is turned off, that is, an attacker cannot exploit the threshold to manipulate the cache metadata.

5 EVALUATION

We evaluate PrefixWall to understand its impact on performance and scalability in multi-tenant LLM serving environments. Our goal is to quantify the performance benefits under realistic workloads, across LLM families and sizes, while examining the overhead introduced by PrefixWall. Specifically, we address the following research questions:

RQ1: **Performance Gains:** How much does PrefixWall accelerate inference compared to existing defenses across different workloads and models (Sections 5.2.1 and 5.2.2)?
RQ2: **Overheads:** What are the latency and resource overheads introduced by PrefixWall (Sections 5.2.1, 5.2.2, and 5.2.4)?
RQ3: **Security:** Can PrefixWall prevent timing leaks and prompt-stealing attacks? (5.2.3)
RQ4: **Sensitivity:** What is the impact of the activation threshold on PrefixWall's performance? (5.2.4)

5.1 Experimental Setup

We evaluate the performance of PrefixWall on a native hardware server equipped with an NVIDIA A100 GPU (40 GB memory). The serving system is vLLM [64] (version 0.8.5) with a block size of 16. Table 2 summarizes the LLM models used for evaluation, detailing their full name, their parameter count, memory footprint, and the remaining GPU memory available for hosting the KV cache.

Evaluation Baselines. We compare PrefixWall against the following baselines, based on the categorization of defenses in Table 1:

- **Prefix Caching:** Enables reuse of common prefixes cached across all users. This approach maximizes reuse and efficiency but offers no isolation or security guarantees. That is, this is vanilla unprotected vLLM with APC active.

Abbreviation	LLM Model	Model Memory	Cache Memory
Gemma3-4B	gemma-3-4b-it [27]	8.6 GB	31.4 GB
Llava-0.5B	llava-onevision-qwen2-0.5b-ov-hf [44]	1.68 GB	38.32 GB
Llava-7B	llava-onevision-qwen2-7b-ov-chat-hf [45]	15.03 GB	24.97 GB
Llama2-7B	Llama-2-7b-chat-hf [48]	12.6 GB	27.4 GB
Llama2-13B	Llama-2-13b-chat-hf [47]	24.3 GB	15.7 GB
Qwen2-2B	Qwen2-VL-2B-Instruct [5]	4.15 GB	35.85 GB
Qwen2.5-3B	Qwen2.5-VL-3B-Instruct [7]	7.16 GB	32.84 GB
Qwen2-7B	Qwen2-VL-7B-Instruct [6]	15.53 GB	24.47 GB
Qwen2.5-7B	Qwen2.5-VL-7B-Instruct [8]	15.63 GB	24.37 GB

Table 2: LLM models used for evaluation, sorted by family and model size.

- **User Cache Isolation:** Allocates dedicated KV cache regions per user to ensure isolation and security [31, 51, 75]. However, this eliminates the benefits of sharing cached common prefixes, reducing efficiency. We implemented this by modifying vLLM by preprending a unique user identifier (a single token) to each user request, which effectively results in isolating all requests of a user from those of other users.
- **PrefixWall:** Our proposed mechanism dynamically isolates common prefixes rather than users, achieving a balance between security and performance.

The baselines are evaluated against two key metrics: time-to-first-token (TTFT) and cache hit rate. TTFT is a widely used metric for assessing LLM inference performance [64], as it reflects the responsiveness of the system in generating the first token. In contrast, cache hit rate indicates the degree to which the shared prefix cache was utilized, providing insight into the effectiveness of the evaluated baselines.

Workloads. For our experiments, we construct 5 multi-user workloads starting from the ShareGPT [9] dataset, which contains real-world LLM requests. Given that we want to understand the effects of prefix caching, the workloads we constructed cover varying level of prefix overlap between requests

Each workload consists of 10 users, each one issuing 100 requests, i.e., 1000 requests overall. The incoming times of the requests follow a Poisson distribution, which is representative of LLM workloads [3, 39, 53, 76]. To create realistic workloads containing private information from different users, we processed the ShareGPT requests as follows:

- For each request in ShareGPT, we asked the BERT [21] model to identify sensitive words in various position in the request and to mask them, i.e., to replace them with `[Mask]`.
- Next, for each masked request, we asked BERT the masked tokens with different candidates, thereby simulating multiple instances of a public request that differ only in user-specific sensitive information.

Then, we constructed the workloads by selecting requests from this augmented dataset. We mix various prompts with varying levels of prefix overlap to capture a wide range of behaviors. Higher overlap means a longer shared prefix, which leads to more cache reuse and better performance. For example, private data at the end of the prompt ("You are a helpful assistant. I want you to write an email to reply to [sensitive information]") creates high overlap, while at the beginning ("My name is [sensitive information]") results in minimal overlap.

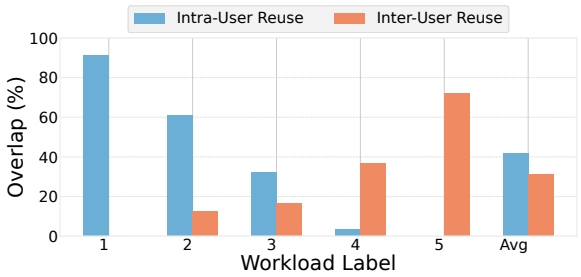


Figure 6: Workload Construction.

To capture a multi-user environment, we vary both the *intra-user* reuse (how often a user reuses a previously cached prefix) and *inter-user* reuse (how much different users share common prefixes). Figure 6 illustrates these dimensions across our constructed workloads. To capture a system under attack, one user acts as an adversary targeting a popular prompt template. The attacker follows the methodology described in Section 4, sending multiple variations of the template with different candidate values at the suspected position of the private information (secret).

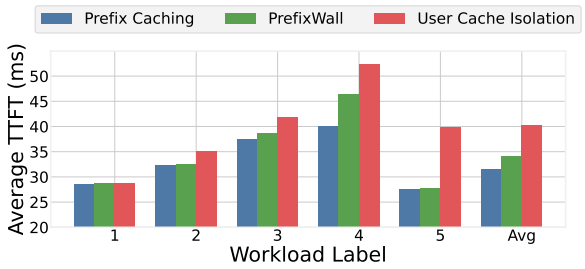
For all workloads and experiments in 5.2, the arrival times of requests are designed so that the system processes 1 request per second, thereby ensuring that the system operates in a scenario where the side-channel is easy to exploit (as shown in 2.2). Note also that, for all experiments in 5.2, the detector component of PrefixWall is always active.

Implementation Details. PrefixWall is implemented on top of vLLM [64] (version 0.8.5) with the V1 engine for multi-tenancy. To extend the KV cache as described in 3.2, we extend the KVCacheBlock class with the two metadata fields. We introduce a new Detector component, as part of the Scheduler that detects malicious requests before scheduling and updates the new metadata fields `attacker_flag` and `owner_id`. We extend the metrics collection of vLLM to support the Activator component which is implemented in the asynchronous LLM execution layer. PrefixWall is extensively documented and will be open-sourced to encourage community adoption and future extensions.

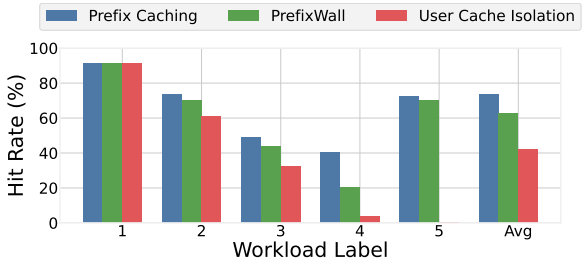
5.2 Inference Performance

Here, we analyze how PrefixWall behaves in terms of inference performance for different kinds of workloads (5.2.1) and models (5.2.2). Next, we empirically validate PrefixWall's behavior on attacker's perceived performance on a concrete workload (5.2.3). We conclude by analysing how the different components of PrefixWall contribute to the overall overhead (5.2.4).

5.2.1 Across Workloads. Figure 7 reports average time-to-first-token (TTFT) and cache hit rate across the five representative workloads (Figure 6) and their aggregate average. The reported values exclude the attacker.



(a) Average TTFT.



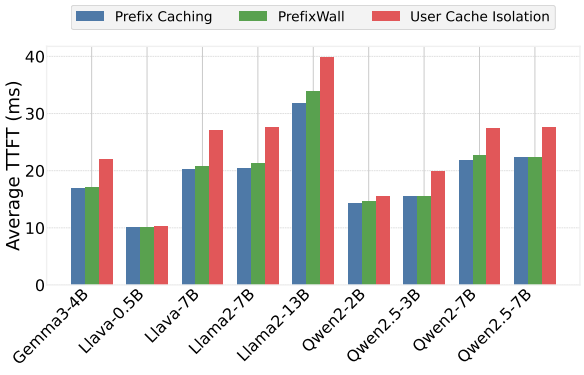
(b) Cache Hit rate.

Figure 7: Comparison of baselines across various workloads.

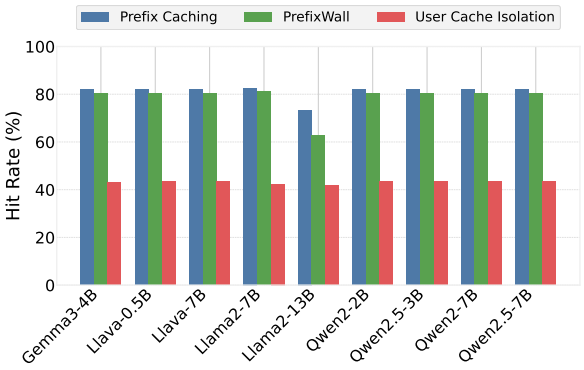
Workloads with high intra-user reuse. Starting with one of the extreme cases, *Workload 1* exhibits high intra-user reuse and zero inter-user reuse, meaning that each user repeats similar prompts but does not share common prefixes with other users. As expected, for this workload all baselines behave similarly. Prefix Caching, PrefixWall, and User Cache Isolation achieve comparable hit rates and TTFT because reuse occurs only within user boundaries. PrefixWall introduces negligible overheads in this scenario, matching the efficiency of isolation while maintaining security. Similarly, *Workloaxd 2* maintains high intra-user reuse but introduces moderate inter-user reuse. As expected, User Cache Isolation suffers lower hit rate and higher TTFT because it misses shared prefixes, while PrefixWall stays very close to Prefix Caching, showing that selective isolation preserves most performance benefits.

Workloads with high inter-user reuse. On the other extreme, *Workload 5* has high inter-user reuse and zero intra-user reuse, meaning that users ask similar prompts to each other but not to themselves. This explains why User Cache Isolation has zero cache hit rate, leading to cache misses and the highest TTFT among all baselines. Most importantly, this workload **stresses the critical path** of PrefixWall by activating the detector on every request due to the high inter-user reuse. However, we see that PrefixWall's lightweight design does not hurt performance: TTFT remains very close to Prefix Caching, and hit rate is similarly high, demonstrating that PrefixWall can enforce security without sacrificing efficiency even under worst-case conditions.

Average Case. Across all workloads (Avg group of bars), PrefixWall performs between the two baselines: it significantly increases hit rate and lowers TTFT compared to User Cache Isolation, while staying within roughly 5–10% of Prefix Caching, which is the best-performing but insecure approach. This demonstrates that



(a) Average TTFT.



(b) Cache hit rate.

Figure 8: Comparison of baselines across various LLMs.

PrefixWall preserves most of the performance benefits of prefix caching while enforcing strong security guarantees, even under diverse workload patterns.

5.2.2 Across LLM Models. Figure 8 shows the average TTFT and cache hit rate of PrefixWall compared to the baselines across the LLM models summarized in Table 2. These models span different families and parameter sizes. The model size directly impacts the available KV cache space on our 40 GB NVIDIA A100 GPU: larger models leave less memory for the cache (Table 2). It also affects inference speed: the smaller the model, the faster the inference and the lower the TTFT [52].

As discussed in Section 2 and shown in Figure 8a, the performance gap between Prefix Caching and User Cache Isolation widens as model size increases and also varies by model family. Across all models, PrefixWall effectively closes this gap, remaining within a 6% margin of Prefix Caching, while significantly outperforming User Cache Isolation. This demonstrates that PrefixWall's selective isolation delivers secure and fast inference. In Figure 8b, cache hit rates are similar across models except for LLaMA-13B. For this large model, the KV cache is very small (Table 2), almost half the size compared to the rest of the models, causing frequent evictions and more cache misses. This limitation also explains the larger TTFT differences across baselines, where even between PrefixWall and

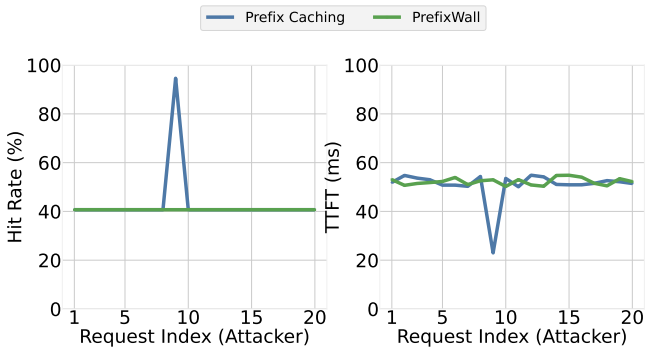


Figure 9: Comparison of prefix caching (unprotected system) and PrefixWall across requests of a single attacker.

Prefix Caching there is a margin of 2 ms. In conclusion, the performance of PrefixWall remains close to Prefix Caching across all LLM models, even under severe memory pressure with large models. This confirms that PrefixWall enables secure LLM serving without unnecessarily reduced performance or unbearable overheads.

5.2.3 Attacker-perceived Performance. Here, we show how PrefixWall behaves from an attacker's perspective. For this, we created a dedicated "attacker workload" consisting of (1) a benign user issuing a request consisting of an otherwise public prompt containing a single piece of secret information in the middle of the prompt, and (2) 20 requests made by an attacking user that tries to recover the secret, i.e., these 20 requests all follow the public prompt while using 20 different values for the candidate secret. Note that the 9-th attacker request is the one guessing the secret correctly.

We run this "attacker workload" on the unprotected Prefix Caching baseline and on PrefixWall. Figure 9 reports the results of this experiment in terms of hit rate and TTFT for the 20 attacker requests (after executing the victim's request first).

For the unprotected Prefix Caching baseline, we observe that both hit rate and TTFT are stable across all requests (reflecting that all requests share the same prompt template), except for the 9th request where we see a clear spike in both the hit rate and TTFT. In particular, hit rate reaches 100%, which means that the attacker's request exactly matches a prompt previously cached, that correlates with a decrease in TTFT. These spikes in hit rate and TTFT are expected since the 9th request is the one correctly guessing the secret, a fact that the attacker can clearly observe through the timing side-channel.

In contrast, for the system secured by PrefixWall, both hit rate and TTFT are uniform across *all* attacker requests. In particular, there is no spike corresponding to the 9th request, i.e., the one where the secret is guessed correctly. That is, an attacker observing TTFT cannot determine which of the 20 requests is the one matching the victim's prompt,. This empirically confirms that PrefixWall successfully closes the timing leak on our "attacker workload", thereby securing the system.

5.2.4 Overheads. 3 summarizes the operational overheads introduced by the two components of PrefixWall in terms of latency and memory resources. In terms of per-request latency, PrefixWall's modifications over vanilla vLLM (i.e., the selective isolation scheme and the activator component) introduce an average per-request overhead of only 0.007 ms (0.004 ms due to the detector component and 0.003 ms due to the activator component). In terms of memory, the metadata introduced by the detector component amount to 32 bytes per cache entry, since each KVCache block was extended with two metadata fields—`owner_id` and `under_attack`—of type `int`.[1] In contrast, the activator component stores 1 floating point value per request in the sliding window.[2]

Component	Time	Memory
Detector	0.004 ms per request	32 bytes per cache entry
Activator	0.003 ms	16 bytes * len(window)

Table 3: Time and memory overheads introduced by the system components of PrefixWall.

5.3 Sensitivity Analysis

The selection of the KDE overlap threshold represents a trade-off between security and performance. With very low thresholds, PrefixWall's selective isolation is active only when the distributions of hits and misses are clearly distinguishable. Conversely, with very high thresholds, selective isolation is active even when the two distributions are not distinguishable and the timing side channel is very difficult to exploit.

To understand the impact of the threshold on the system's performance, we run PrefixWall with varying KDE thresholds (from 0% to 100%) for the LLama2-13B model and Workload 4, which exhibits high reuse, making it suitable for studying the effect of the threshold. 10 reports the results of our experiments, together with the values associated with the different baselines: Prefix Caching in blue, User Cache Isolation in red, and PrefixWall when the detector is always active (i.e., the configuration from 5.2) in green. The results align with expectations. For low threshold values (close to 0%), the system performs comparably to Prefix Caching for the same workload since selective-isolation is almost always disabled, i.e., PrefixWall behaves as the default insecure system. For high threshold (close to 100%), the system achieves the hit rate and performance of Selective Cache Isolation. In general, increasing the KDE threshold increases the security guarantees—because more requests are selectively isolated—and introduces more overhead, so the activator component provides the system administrator with a way of effectively balancing performance and security. We also note that, consistently with results from 5.2, for all KDE values, PrefixWall outperforms the User Cache Isolation baseline.

6 RELATED WORK

Optimizations of LLM Serving Systems. Recent system-level optimizations use *approximate* techniques, such as modifying the attention mechanism [20, 35, 42, 68, 77], reusing [25, 26, 69], or compressing the KV cache [23, 40, 43, 56, 60, 61]. However, these techniques sacrifice accuracy for faster inference, which may not be acceptable for all use cases. On the other hand, *exact* techniques aim to reduce the latency by efficiently managing the KV cache [39,

[1] Since vLLM is implemented in Python, adding two `int` fields (as Python objects) to the KVCache class results in an increase of 32 bytes.
[2] Again, a floating point value in Python takes up to 16 bytes.

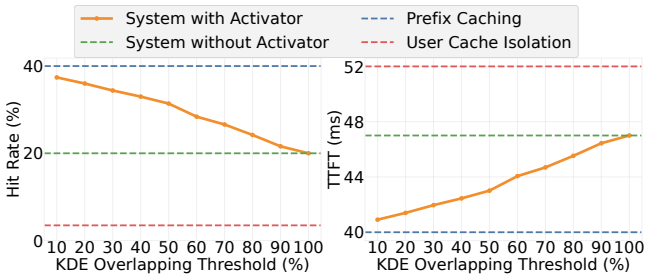


Figure 10: Comparison of hit rate and TTFT as a function of KDE overlapping threshold.

55, 72], sharing prefixes [36, 37, 41, 71], or optimizing scheduling decisions [1–3, 15, 33, 73]. Other approaches attempt to mitigate the impact of long-context processing by disaggregating the prefill and decode phases of inference [53, 76] or rescheduling requests to reduce memory fragmentation [59].

Side-channels and performance optimizations. Performance optimizations work by improving resource consumption (e.g., timing, memory, energy) on the average case. Thus, they introduce variations in the resource usage profile that have been shown, time and again, to lead to side-channels that malicious attackers can exploit to leak sensitive information. At the hardware level, historically the first side-channel attacks exploited leaks that resulted from physical side-effects, such as power consumption or electromagnetic emanations, which require physical access to the device under attack. More recently, however, side channels have been shown to exist due to microarchitectural components that are shared between processes, such as the CPU cache [70], the DRAM [54], branch prediction units [22], the TLB [28], execution ports [4], and the ring interconnect [50]. Side channels are not limited to hardware, though. There have been many side-channels arising from optimizations in software like browsers [57, 63], operating systems [32], or even communication protocols [46, 66].

Timing Side-Channel Attacks in LLM Serving Systems. Recent research has shown that performance optimizations in LLM serving systems, such as KV caching and prefix reuse, introduce exploitable timing side channels. Song et al. [58] were among the first to identify these issues, demonstrating that cache hits and misses produce measurable latency differences that can be leveraged to reconstruct user prompts. Building on this insight, Zheng et al. [75] proposed practical probing strategies that exploit these timing variations to steal sensitive input data in multi-tenant environments. More recently, Gu et al. [31] highlighted privacy risks in commercial APIs, revealing that shared caches can leak user prompts even without direct memory access.

Beyond cache-based optimizations, other works have explored timing side-channels introduced by inference optimizations. For example, Carlini and Nasr [14] show that techniques such as speculative decoding create data-dependent timing variations that can be exploited by a network adversary to infer conversation topics or sensitive information over encrypted channels. In contrast, Wu et al. [67] do not rely on timing differences; instead, it reconstructs prompts token by token by exploiting scheduling policies (e.g., Longest Prefix Match) and observing changes in serving order.

Collectively, these works establish the feasibility and severity of prompt-stealing attacks via timing side-channels, underscoring the need for robust defenses in LLM serving systems.

Further defenses against prompt-stealing attacks We already provided a review of existing defenses against prompt-stealing attacks that focus on mitigating timing leaks through APC in 2.4. Here, we analyze further defenses proposed in the literature. To mitigate prompt-stealing attacks through APC in vLLM, Song et al. [58] proposed to increase the number of tokens per cache entry. Although this does not prevent the leak, it makes more difficult for attackers to guess correctly the secret, since now they have to guess all tokes in an entry all at once.

7 SUMMARY

This paper builds PrefixWall, a system that secures LLM serving against timing side-channel attacks introduced by Automatic Prefix Caching (APC) without sacrificing performance. PrefixWall monitors cache access patterns across users, flags suspicious reuse of cached prefixes, and isolates later paths of flagged prefixes using minimal extensions to KV cache entries. Evaluation shows that CacheSentinel achieves real-time protection against prompt-stealing attacks and achieves high performance and scalability, effectively closing the gap between unsecure APC and current heavy-handed mitigations.

REFERENCES

[1] Amey Agrawal, Junda Chen, Íñigo Goiri, Ramachandran Ramjee, Chaojie Zhang, Alexey Tumanov, and Esha Choukse. 2024. Mnemosyne: Parallelization Strategies for Efficiently Serving Multi-Million Context Length LLM Inference Requests Without Approximations. arXiv:2409.17264 [cs.LG] https://arxiv.org/abs/2409.17264

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2025. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 7, 18 pages.

[3] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. arXiv:2308.16369 [cs.LG] https://arxiv.org/abs/2308.16369

[4] Alejandro Cabrera Aldaya, Billy Bob Brumley, Sohaib ul Hassan, Cesar Pereida García, and Nicola Tuveri. 2019. Port Contention for Fun and Profit. In *Proceedings of the 40th IEEE Symposium on Security and Privacy (S&P '19)*. IEEE.

[5] Alibaba Cloud. 2024. *Qwen2-VL 2B Instruct Model.* https://huggingface.co/Qwen/Qwen2-VL-2B-Instruct Available on Hugging Face Hub.

[6] Alibaba Cloud. 2024. *Qwen2-VL 7B Instruct Model.* https://huggingface.co/Qwen/Qwen2-VL-7B-Instruct Available on Hugging Face Hub.

[7] Alibaba Cloud. 2024. *Qwen2.5-VL 3B Instruct Model.* https://huggingface.co/Qwen/Qwen2.5-VL-3B-Instruct Available on Hugging Face Hub.

[8] Alibaba Cloud. 2024. *Qwen2.5-VL 7B Instruct Model.* https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct Available on Hugging Face Hub.

[9] anon8231489123. 2023. ShareGPT Vicuna unfiltered. https://huggingface.co/datasets/anon8231489123/ShareGPT%20Vicuna%20unfiltered. Dataset on Hugging Face.

[10] F. Bang. 2023. Gptcache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*. 212–218.

[11] BelleGroup. 2023. Multiturn Chat 0.8M. https://huggingface.co/datasets/BelleGroup/multiturn%20chat%200.8M. Dataset on Hugging Face.

[12] Andrew Bortz and Dan Boneh. 2007. Exposing private information by timing web applications. In *Proceedings of the 16th International Conference on World Wide Web* (Banff, Alberta, Canada) *(WWW '07)*. Association for Computing Machinery, New York, NY, USA, 621–628. https://doi.org/10.1145/1242572.1242656

[13] Tom Brown, Benjamin Mann, Nick Ryder, et al. 2020. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems* 33 (2020), 1877–1901.

[14] Nicholas Carlini and Milad Nasr. 2024. Remote Timing Attacks on Efficient Language Model Inference. arXiv:2410.17175 [cs.CR] https://arxiv.org/abs/2410.17175

[15] Shaoyuan Chen, Yutong Lin, Mingxing Zhang, and Yongwei Wu. 2024. Efficient and Economic Large Language Model Inference with Attention Offloading. arXiv:2405.01814 [cs.LG] https://arxiv.org/abs/2405.01814

[16] Kexin Chu, Zecheng Lin, Dawei Xiang, Zixu Shen, Jianchang Su, Cheng Chu, Yiwei Yang, Wenhui Zhang, Wenfei Wu, and Wei Zhang. 2025. Selective KV-Cache Sharing to Mitigate Timing Side-Channels in LLM Inference. arXiv:2508.08438 [cs.CR] https://arxiv.org/abs/2508.08438

[17] Rasmus Dahlberg and Tobias Pulls. 2023. Timeless Timing Attacks and Preload Defenses in Tor's DNS Cache. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 2635–2652. https://www.usenix.org/conference/usenixsecurity23/presentation/dahlberg

[18] DeepMind. 2025. *Gomini*. https://deepmind.google/technologies/gemini/

[19] DeepSeek. 2024. *DeepSeek API Docs: DeepSeek API Introduces Context Caching on Disk, Cutting Prices by an Order of Magnitude*. https://api-docs.deepseek.com/news/news0802/ Accessed: 2025-07-17.

[20] Aditya Desai, Shuo Yang, Alejandro Cuadron, Ana Klimovic, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2024. HashAttention: Semantic Sparsity for Faster Inference. arXiv:2412.14468 [cs.LG] https://arxiv.org/abs/2412.14468

[21] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 4171–4186. https://doi.org/10.48550/arXiv.1810.04805

[22] Dmitry Evtyushkin, Ryan Riley, Nael B. Abu-Ghazaleh, and Dmitry Ponomarev. 2018. BranchScope: A New Side-Channel Attack on Directional Branch Predictor. In *Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '18)*. ACM.

[23] Qichen Fu, Minsik Cho, Thomas Merth, Sachin Mehta, Mohammad Rastegari, and Mahyar Najibi. 2024. LazyLLM: Dynamic Token Pruning for Efficient Long Context LLM Inference. arXiv:2407.14057 [cs.CL] https://arxiv.org/abs/2407.14057

[24] V. Gallego. 2024. Configurable Safety Tuning of Language Models with Synthetic Preference Data. (2024). Preprint.

[25] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2025. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *Proceedings of the 2024 USENIX Conference on Usenix Annual Technical Conference* (Santa Clara, CA, USA) *(USENIX ATC'24)*. USENIX Association, USA, Article 7, 16 pages.

[26] In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. arXiv:2311.04934 [cs.CL] https://arxiv.org/abs/2311.04934

[27] Google DeepMind. 2025. *Gemma 3 4B Instruct Model*. https://huggingface.co/google/gemma-3-4b-it Available on Hugging Face Hub.

[28] Ben Gras, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2018. Translation Leak-aside Buffer: Defeating Cache Side-channel Protections with TLB Attacks. In *Proceedings of the 27th USENIX Security Symposium (USENIX Security '18)*. USENIX Association.

[29] Ben Gras, Kaveh Razavi, Erik Bosman, Herbert Bos, and Cristiano Giuffrida. 2017. ASLR on the Line: Practical Cache Attacks on the MMU. In *NDSS*. Paper=https://download.vusec.net/papers/anc_ndss17.pdfSlides=https://vusec.net/wp-content/uploads/2016/11/TalkGras.pdfWeb=https://www.vusec.net/projects/ancCode=https://github.com/vusec/revancPress=https://goo.gl/KL4Bta

[30] Daniel Gruss, Erik Kraft, Trishita Tiwari, Michael Schwarz, Ari Trachtenberg, Jason Hennessey, Alex Ionescu, and Anders Fogh. 2019. Page Cache Attacks. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security* (London, United Kingdom) *(CCS '19)*. Association for Computing Machinery, New York, NY, USA, 167–180. https://doi.org/10.1145/3319535.3339809

[31] Chenchen Gu, Xiang Lisa Li, Rohith Kuditipudi, Percy Liang, and Tatsunori Hashimoto. 2025. Auditing Prompt Caching in Language Model APIs. arXiv:2502.07776 [cs.CL] https://arxiv.org/abs/2502.07776

[32] Marcus Hähnel, Weidong Cui, and Marcus Peinado. 2017. High-resolution side channels for untrusted operating systems. In *Proceedings of the 2017 USENIX Conference on Usenix Annual Technical Conference* (Santa Clara, CA, USA) *(USENIX ATC '17)*. USENIX Association, USA, 299–312.

[33] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He. 2024. DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference. arXiv:2401.08671 [cs.PF] https://arxiv.org/abs/2401.08671

[34] Henry F. Inman and Edwin L. Bradley. 1989. The overlapping coefficient as a measure of agreement between probability distributions. *Communications in Statistics-Theory and Methods* 18, 10 (1989), 3851–3874. https://doi.org/10.1080/03610928908830127

[35] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. MInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention. arXiv:2407.02490 [cs.CL] https://arxiv.org/abs/2407.02490

[36] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. 2025. RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation. *ACM Trans. Comput. Syst.* 44, 1, Article 2 (Nov. 2025), 27 pages. https://doi.org/10.1145/3768628

[37] Jordan Juravsky, Bradley Brown, Ryan Ehrlich, Daniel Y. Fu, Christopher Ré, and Azalia Mirhoseini. 2024. Hydragen: High-Throughput LLM Inference with Shared Prefixes. arXiv:2402.05099 [cs.LG] https://arxiv.org/abs/2402.05099

[38] David Kohlbrenner and Hovav Shacham. 2016. Trusted browsers for uncertain times. In *Proceedings of the 25th USENIX Conference on Security Symposium* (Austin, TX, USA) *(SEC'16)*. USENIX Association, USA, 463–480.

[39] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[40] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 155–172. https://www.usenix.org/conference/osdi24/presentation/lee

[41] Jieyu Lin, Sai Qian Zhang, and Alberto Leon-Garcia. 2024. sLLM: Accelerating LLM Inference using Semantic Load Balancing with Shared Memory Data Structures. In *2024 25th International Symposium on Quality Electronic Design (ISQED)*. 1–6. https://doi.org/10.1109/ISQED60706.2024.10528703

[42] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023. Ring Attention with Blockwise Transformers for Near-Infinite Context. arXiv:2310.01889 [cs.CL] https://arxiv.org/abs/2310.01889

[43] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) *(ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 38–56. https://doi.org/10.1145/3651890.3672274

[44] LLaVA Team. 2024. *LLaVA-OneVision Qwen2 0.5B (OV-HF)*. https://huggingface.co/llava-onevision-qwen2-0.5b-ov-hf Available on Hugging Face Hub.

[45] LLaVA Team. 2024. *LLaVA-OneVision Qwen2 7B OV-Chat (HF)*. https://huggingface.co/llava-onevision-qwen2-7b-ov-chat-hf Available on Hugging Face Hub.

[46] Aastha Mehta, Mohamed Alzayat, Roberta De Viti, Björn B. Brandenburg, Peter Druschel, and Deepak Garg. 2022. Pacer: Comprehensive Network Side-Channel Mitigation in the Cloud. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 2819–2838. https://www.usenix.org/conference/usenixsecurity22/presentation/mehta

[47] Meta AI. 2023. *Llama 2 13B Chat Model*. https://huggingface.co/meta-llama/Llama-2-13b-chat-hf Available on Hugging Face Hub.

[48] Meta AI. 2023. *Llama 2 7B Chat Model*. https://huggingface.co/meta-llama/Llama-2-7b-chat-hf Available on Hugging Face Hub.

[49] OpenAI. 2024. *Prompt Caching: Reduce Latency and Cost with Prompt Caching*. https://platform.openai.com/docs/guides/prompt-caching Accessed: 2025-07-17.

[50] Riccardo Paccagnella, Licheng Luo, and Christopher W. Fletcher. 2021. Lord of the Ring(s): Side Channel Attacks on the CPU On-Chip Ring Interconnect Are Practical. In *Proceedings of the 30th USENIX Security Symposium (USENIX Security '21)*. USENIX Association.

[51] Zixuan Pang, Wenhao Wang, and Yong Liao. 2024. Cache Partitioning for Mitigating Timing Side-Channel Attacks in LLM Serving Systems. In *2024 6th International Conference on Frontier Technologies of Information and Computer (ICFTIC)*. 1238–1245. https://doi.org/10.1109/ICFTIC64248.2024.10913329

[52] Konstantinos Papaioannou and Thaleia Dimitra Doudali. 2024. The Importance of Workload Choice in Evaluating LLM Inference Systems. In *Proceedings of the 4th Workshop on Machine Learning and Systems* (Athens, Greece) *(EuroMLSys '24)*. Association for Computing Machinery, New York, NY, USA, 39–46. https://doi.org/10.1145/3642970.3655823

[53] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. https://doi.org/10.1109/ISCA59077.2024.00019

[54] Peter Pessl, Daniel Gruss, Clémentine Maurice, Michael Schwarz, and Stefan Mangard. 2016. DRAMA: Exploiting DRAM Addressing for Cross-CPU Attacks. In *Proceedings of the 25th USENIX Security Symposium (USENIX Security '16)*. USENIX Association.

[55] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-Centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 155–170.

[56] Zhenmei Shi, Yifei Ming, Xuan-Phi Nguyen, Yingyu Liang, and Shafiq Joty. 2025. Discovering the Gems in Early Layers: Accelerating Long-Context LLMs with 1000x Input Token Reduction. https://openreview.net/forum?id=9iN8p1Xwtg

[57] Peter Snyder, Soroush Karami, Arthur Edelstein, Benjamin Livshits, and Hamed Haddadi. 2023. Pool-party: exploiting browser resource pools for web tracking. In *Proceedings of the 32nd USENIX Conference on Security Symposium* (Anaheim, CA, USA) *(SEC '23)*. USENIX Association, USA, Article 397, 15 pages.

[58] Linke Song, Zixuan Pang, Wenhao Wang, Zihao Wang, XiaoFeng Wang, Hongbo Chen, Wei Song, Yier Jin, Dan Meng, and Rui Hou. 2025. The Early Bird Catches the Leak: Unveiling Timing Side Channels in LLM Serving Systems. arXiv:2409.20002 [cs.CR] https://arxiv.org/abs/2409.20002

[59] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 173–191. https://www.usenix.org/conference/osdi24/presentation/sun-biao

[60] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. 2024. ShadowKV: KV Cache in Shadows for High-Throughput Long-Context LLM Inference. arXiv:2410.21465 [cs.LG] https://arxiv.org/abs/2410.21465

[61] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. QUEST: query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) *(ICML'24)*. JMLR.org, Article 1955, 11 pages.

[62] Hugo Touvron et al. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971* (2023).

[63] Pepe Vila and Boris Kopf. 2017. Loophole: Timing Attacks on Shared Event Loops in Chrome. In *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, Vancouver, BC, 849–864. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/vila

[64] vLLM Team. 2024. vLLM: High-Throughput Serving for Large Language Models. https://github.com/vllm-project/vllm.

[65] vLLM Team. 2025. Automatic Prefix Caching in vLLM. https://docs.vllm.ai/en/latest/features/automatic_prefix_caching/.

[66] Charles V. Wright, Lucas Ballard, Scott E. Coull, Fabian Monrose, and Gerald M. Masson. 2008. Spot Me if You Can: Uncovering Spoken Phrases in Encrypted VoIP Conversations. In *2008 IEEE Symposium on Security and Privacy (sp 2008)*. 35–49. https://doi.org/10.1109/SP.2008.21

[67] Guanlong Wu, Zheng Zhang, Yao Zhang, Weili Wang, Jianyu Niu, Ye Wu, and Yinqian Zhang. 2025. I Know What You Asked: Prompt Leakage via KV-Cache Sharing in Multi-Tenant LLM Serving. In *Proceedings of the 2025 Network and Distributed System Security (NDSS) Symposium. San Diego, CA, USA*.

[68] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. arXiv:2309.17453 [cs.CL] https://arxiv.org/abs/2309.17453

[69] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) *(EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 94–109. https://doi.org/10.1145/3689031.3696098

[70] Yuval Yarom and Katrina Falkner. 2014. FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack. In *Proceedings of the 23rd USENIX Security Symposium (USENIX Security '14)*. USENIX Association.

[71] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. ChunkAttention: Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 11608–11620. https://doi.org/10.18653/v1/2024.acl-long.623

[72] Lingfan Yu, Jinkun Lin, and Jinyang Li. 2025. Stateful Large Language Model Serving with Pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) *(EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 144–158. https://doi.org/10.1145/3689031.3696086

[73] Siyan Zhao, Daniel Israel, Guy Van den Broeck, and Aditya Grover. 2024. Prepacking: A Simple Method for Fast Prefilling and Increased Throughput in Large Language Models. arXiv:2404.09529 [cs.LG] https://arxiv.org/abs/2404.09529

[74] L. Zheng, L. Yin, Z. Xie, C. L. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, et al. 2024. SGLang: Efficient Execution of Structured Language Model Programs. *Advances in Neural Information Processing Systems* 37 (2024), 62557–62583.

[75] Xinyao Zheng, Husheng Han, Shangyi Shi, Qiyan Fang, Zidong Du, Xing Hu, and Qi Guo. 2024. InputSnatch: Stealing Input in LLM Services via Timing Side-Channel Attacks. arXiv:2411.18191 [cs.CR] https://arxiv.org/abs/2411.18191

[76] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 193–210. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

[77] Qianchao Zhu, Jiangfei Duan, Chang Chen, Siran Liu, Xiuhong Li, Guanyu Feng, Xin Lv, Xiao Chuanfu, Dahua Lin, and Chao Yang. 2025. SampleAttention: Near-Lossless Acceleration of Long Context LLM Inference with Adaptive Structured Sparse Attention. In *Eighth Conference on Machine Learning and Systems*. https://openreview.net/forum?id=RuZ80yl71h